

Curso Inicial Front-End

Clase 13: Control de

Flujo y Estructuras de

Repetición

Módulo 2: Dominio de JavaScript y Lógica de Programación



Presentación

Programar es aprender a tomar decisiones. En esta clase conoceremos los **Operadores Lógicos**, las herramientas que permiten que tu código piense y reaccione según diferentes condiciones. Dominaremos las **Tablas de Verdad** y entenderemos el concepto de **Truthy y Falsy**, clave para evitar errores inesperados en el flujo de tus aplicaciones.



Objetivos

- Dominar el uso de operadores lógicos (`&&`, `||`, `!`) y de comparación.
- Comprender y aplicar tablas de verdad para resolver condiciones complejas.
- Identificar valores **Truthy** y **Falsy** para escribir código más limpio y robusto.



Bloques Temáticos

- Tema 1: Lógica condicional: if/else y switch.
 - Coerción y Comparación: El Peligro Oculto de JS
 - Operadores Lógicos: Las Tablas de la Verdad
 - Estructuras de Control: Desviando el Río
 - Truthy y Falsy: ¿Qué es "Verdad" para la computadora?
 - El Operador Ternario (El `if` de bolsillo)
- Tema 2: Bucles: for, while y do/while.
 - ¿Qué es un Bucle?
 - for, while y do/while.

En la clase anterior aprendimos a crear "cajitas" (variables) y guardar datos en ellas. Pero un programa que solo guarda datos es inútil. Hoy aprenderemos a enseñarle a la computadora a comparar esos datos y a **tomar decisiones inteligentes** por su cuenta, desviando el camino de la ejecución según las reglas que nosotros le pongamos.

En lenguaje simple: Programar es darle una lista de condiciones a la computadora para que sepa qué camino tomar. Si se cumple una condición, el motor puede dejar de leer el resto para ahorrar energía (cortocircuito).

Traducción Técnica: El control de flujo se basa en la evaluación de expresiones en contextos booleanos. **JavaScript** utiliza el algoritmo **ToBoolean** (especificación ECMAScript) para convertir cualquier valor. El encadenamiento

lógico mediante `&&` y `||` utiliza la **evaluación de cortocircuito**, donde el motor devuelve el primer operando que determine el resultado final, permitiendo patrones como la ejecución condicional en una sola línea.

 **Recurso Oficial:** [MDN Web Docs - Expresiones y Operadores](https://developer.mozilla.org/es/docs/JavaScript/Guide/Expressions_and_Operators)

Tema 1: Lógica condicional: if/else y switch.

1. Coerción y Comparación: El Peligro Oculto de JS

Para que la computadora decida algo, primero debe comparar dos valores. Pero aquí entra en juego un concepto vital de **JavaScript** llamado **Coerción** (que significa forzar la conversión de un tipo de dato a otro).

- **Coerción Implícita (El comportamiento automático y peligroso):** Ocurre cuando **JS**, en un intento desesperado por ayudarte a que el código no explote, cambia los tipos de datos por debajo de la mesa sin avisarte.
 - Si haces `5 + "5"`, **JS** convierte el primer 5 a texto y el resultado es `"55"`.
 - Pero si haces `5 - "5"`, **JS** convierte el texto a número y el resultado es `0`. ¡Una locura matemática!
- **Coerción Explícita (Lo correcto y profesional):** Ocurre cuando **TÚ** como programador le das la orden estricta a **JS** de transformar el dato, tomando el control absoluto.

- `Number("5") + 5` = Devuelve matemáticamente `10`.
- `String(5) + "5"` = Devuelve el texto concatenado `"55"`.

Comparativa: Igualdad Débil (`==`) vs Igualdad Estricta (`===`) 🥊

A la hora de preguntarle al código "¿Es este valor igual a este otro?", tenemos dos herramientas. Elegir la incorrecta te costará tu trabajo.

Operador de Igualdad	Nombre Técnico	¿Usa Coerción Implícita?	Si le pregunto: <code>5 == "5"</code>	Regla Industrial
<code>==</code> (Dos iguales)	Igualdad Débil	Sí. Intenta adivinar y transforma los tipos antes de comparar.	JS miente y te dice que es <code>true</code> .	✗ NUNCA lo uses. Genera los peores bugs de la industria.
<code>===</code> (Tres iguales)	Igualdad Estricta	No. Compara severamente el Valor y también el Tipo de Dato exacto.	JS detecta que uno es número y otro texto, responde <code>false</code> .	✓ Úsalo SIEMPRE. Es el estándar absoluto en programación.

🛑 **Checkpoint de Comprensión:** Estás programando el inicio de sesión de un banco. Tienes un operador `!=` (Desigualdad Débil) y un `!==` (Desigualdad Estricta). Si un hacker inyecta el número `0` en la contraseña, y la contraseña real era el texto `"0"`, ¿qué operador impediría correctamente el acceso al evaluar el tipo de dato?

2. Operadores Lógicos: Las Tablas de la Verdad

A veces una sola comparación no alcanza. Imagina que para entrar a una sala VIP en un club nocturno, el código debe validar dos cosas a la vez: ser mayor de edad **Y** tener una entrada válida.

- **AND (&&)** - **La Conjunción (El Guardia Estricto)**: Lee de izquierda a derecha. Exige que **TODAS** las condiciones conectadas sean verdaderas (**true**). *Comportamiento técnico (Cortocircuito)*: Si el código lee que la primera condición es **false**, automáticamente detiene la lectura y rechaza todo. Ni siquiera se gasta en leer la segunda condición.
 - **true && true** =  Pasa (**true**)
 - **true && false** =  Se rechaza (**false**)
- **OR (||)** - **La Disyunción (El Guardia Relajado)**: Exige que **AL MENOS UNA** condición sea verdadera. *Comportamiento técnico (Cortocircuito)*: Si el código lee que la primera condición ya es **true**, frena ahí, abre las puertas y aprueba todo sin importarle lo que diga el resto.
 - **false || true** =  Pasa (**true**)
 - **false || false** =  Se rechaza (**false**)
- **NOT (!)** - **El Operador de Negación (El Día del Revés)**: Se pone justo antes de una variable para invertir su valor lógico instantáneamente. **!true** se convierte en **false**.

3. Estructuras de Control: Desviando el Río



El bloque `if`, `else if` y `else`

Es el mecanismo principal de toma de decisiones. Significa literalmente: "Si pasa A, haz esto; si no, si pasa B, haz aquello; y si ninguna pasa, haz esto por defecto".

Anatomía del código: * Observa los **paréntesis** `()`: Adentro va exclusivamente la evaluación lógica o matemática. * Observa las **llaves** `{}`: Adentro va el bloque de código de acción que solo se ejecutará si los paréntesis dieron como resultado `true`.

```
let horaDelDia = 14;

// El motor lee: ¿Es 14 menor que 12? Falso. Salta al siguiente bloque.
if (horaDelDia < 12) {
  console.log("Buenos días");
}
// El motor lee: ¿Es 14 menor que 19? ¡Verdadero! Entra en las llaves y se
detiene el análisis.
else if (horaDelDia < 19) {
  console.log("Buenas tardes");
}
// El 'else' final no lleva paréntesis porque atrapa todo lo que sobró.
else {
  console.log("Buenas noches");
}
```

El `switch`: El despachador múltiple

Cuando necesitas comparar exactamente **una sola variable** contra muchos valores posibles fijos, hacer un `if/else` gigante es desprolijo. Para eso usamos el `switch`:

```
let planElegido = "Premium";

switch (planElegido) {
  case "Basico":
    console.log("Tienes 10GB de espacio");
    break; // ¡VITAL! El break corta la ejecución para que no lea el
    caso de abajo.
  case "Premium":
    console.log("Tienes espacio ilimitado");
    break;
  default: // Equivalente al 'else'
    console.log("Plan no reconocido");
}
```

4. Truthy y Falsy: ¿Qué es "Verdad" para la computadora? 🤔

En **JavaScript**, los valores primitivos (como un número suelto o un texto vacío) tienen "sentimientos" booleanos ocultos cuando los obligas a entrar en los paréntesis de un `if ()`. **JS** los convertirá a `true` o `false` a la fuerza.

- **Valores Falsy (El núcleo del mal):** Existen 6 valores concretos en **JS** que, al ser forzados, se evalúan matemáticamente como `false`. Memorízalos: `0`, `""` (texto totalmente vacío), `null`, `undefined`, `NaN`, y por supuesto, `false`.
- **Valores Truthy (El resto del universo):** Absolutamente cualquier otro valor se evalúa como `true`. ¡Cuidado! Incluso un espacio en blanco `" "` o un objeto vacío `{}` son considerados como verdaderos.

La Tabla de Verdad (And & Or)

Para dominar la lógica, debes memorizar cómo se combinan los valores booleanos:

Valor A	Valor B	Resultado A && B (AND)	Resultado A B (OR)
true	true	true ✓	true ✓
true	false	false ✗	true ✓
false	true	false ✗	true ✓
false	false	false ✗	false ✗

Valores Falsy & Truthy

En **JavaScript**, no solo `true` y `false` activan un `if`. Todo valor tiene una carga booleana.

Valores Falsy (Se evalúan como `false`):

1. `false`
2. `0` (Cero)
3. `""` (String vacío)
4. `null`
5. `undefined`
6. `NaN` (Not a Number)

Valores Truthy (Se evalúan como `true`):

1. "Cualquier texto" (Incluso `"false"` o `" "`).
2. `3.14` (Cualquier número distinto de cero).
3. `[]` (Arrays vacíos).

4. `{}` (Objetos vacíos).

5. `true`.

5. El Operador Ternario (El `if` de bolsillo)

Los profesionales aman este operador por su velocidad de escritura. Es perfecto para asignar el valor de una variable tomando una decisión rápida en una sola línea de código, sin tener que escribir las llaves molestas del `if`.

Anatomía: `condición ? si_es_verdadero : si_es_falso`

```
// En lugar de hacer 5 líneas de if/else, lo resolvemos en una:  
const clima = "soleado";  
const mensajeAlUsuario = (clima === "soleado") ? "Vamos a la playa" :  
"Lleva paraguas";
```

IMPORTANT

Regla de uso Industrial: Nunca anides un ternario dentro de otro ternario. Si tu lógica requiere más de dos opciones (verdadero o falso), el operador ternario se vuelve ilegible y debes usar un `if/else` tradicional.

Tema 2: Bucles: for, while y do/while.

1. ¿Qué es un Bucle? (Analogía del Mundo Real)

Imagina tu rutina de la mañana

- Levantarte
- Ducharte
- Desayunar
- Ir al trabajo

Repites estos pasos cada día mientras sea día laboral.

Eso es un **bucle**: repetir acciones mientras se cumple una condición.



Analogía: Correr Vueltas

Piensa en correr vueltas en una pista:

- **Condición:** Mientras no hayas completado 5 vueltas
- **Acción:** Correr una vuelta
- **Actualización:** Contar la vuelta completada
- **Repites** la acción (correr) hasta cumplir la condición (5 vueltas).



Analogía: Leer un Libro

- **Cuando lees un libro:**
- **Condición:** Mientras haya páginas por leer

- **Acción:** Leer una página
- **Actualización:** Pasar a la siguiente página
- **Repites** la acción (leer) hasta cumplir la condición (terminar el libro).

¿Qué es un Bucle en Programación?

Un bucle (**loop**) permite ejecutar un bloque de código repetidamente mientras se cumple una condición.

En términos simples: Es como decirle a la computadora "haz esto una y otra vez **hasta** que algo cambie".

Bucle FOR

***¿Cuándo usarlo?** Cuando conoces el número de iteraciones o quieres control completo.*

Sintaxis:

```
for (inicialización; condición; actualización) {  
    // Bloque de código  
}
```

Ejemplo básico para contar desde 0 hasta 4:

```
for (let i = 0; i < 5; i++) {  
    console.log(i);  
}
```

```
}
```

Los bucles son la herramienta perfecta para procesar arrays (**listas de datos**). En lugar de escribir el mismo código para cada elemento uno por uno, el bucle **automatiza** la tarea. No importa si la lista tiene 5 o 5 millones de elementos: el bucle la recorre por completo en milisegundos y sin que tengas que duplicar líneas de código.

```
Iterar arrays, muestra cada fruta por consola:  
  
let frutas = ["uva", "manzana", "pera", "mandarina", "naranja"];  
  
for (let i = 0; i < frutas.length; i++) {  
  console.log(frutas[i]);  
}
```

While

¿Cuándo usarlo? Cuando no conoces el número exacto de iteraciones, pero sabes la condición.

⚠ ¡CUIDADO CON EL BUCLE INFINITO! A diferencia del bucle for (donde configuras el inicio, la condición y el paso en una sola línea), el bucle while es más libre y peligroso. Si te olvidas de modificar manualmente dentro del código la variable que controla la condición, esta nunca cambiará y crearás un bucle infinito que puede colgar el navegador o la consola.

Sintaxis:

```
while (condición) {  
    // Bloque de código  
}
```

Ejemplo:

```
let contador = 0;  
let frutas = ["uva", "manzana", "pera", "mandarina", "naranja"];  
  
while (contador < frutas.length) {  
    console.log(`Fruta ${contador + 1}: ${frutas[contador]}`);  
    contador++; // ⚠ Actualizar para evitar loop infinito  
}
```

Do...while

¿Cuándo usarlo? Cuando quieres que el código se ejecute al menos una vez antes de comprobar la condición.

Sintaxis:

```
do {  
    // Bloque de código (se ejecuta al menos una vez)  
} while (condición);
```

Ejemplo, si el numero ingresado es 10 o menor se termina la operación, de lo contrario sigue ejecutándose:

```
let numero;  
do {
```

```
numero = prompt("Ingresa un número mayor a 10:");  
} while (numero <= 10);
```

Documentación Oficial (Referencias)

- **JS:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **JavaScript:** <https://developer.mozilla.org/es/docs/Web/JavaScript>